cost of hashing the character, plus the cost of resolving key conflicts), but also the cost for resizing the dictionary when new edges are added.

6.3 Radix Tries

To overcome these issues with tries, a few alternatives have been developed: the *ternary search trie* (TST), which trades-off lower memory usage for worse running time, or the *radix trie*, just to name a few.

While TSTs improve the space requirements to store links, and frees us from worrying about platform-specific implementations to optimize how we store edges, the number of nodes we need to create is still of the order of magnitude of the number of characters contained in the whole corpus stored, `O(n*m)` for `n` words of average length `m`.

In tries, most of the nodes don't store keys, and are just hops on a path between a key and the ones that extend it. Most of these hops are necessary, but when we store long words, they tend to produce long chains of internal nodes, each with just one child. As we have seen in section 6.2.1, this is the main reason tries need too much space, sometimes more than BSTs.

Figure 6.11 shows an example of a trie: nothing special, just a small, regular trie. We can see that intermediate nodes always have children (assuming we prune dangling branches after deleting keys), sometimes just one child, sometimes more.